

Project Technical Report

Title

An Agile-Governed Automated Release Pipeline for Dev-to-Production Deployment of a Cloud Web Application on AWS EC2

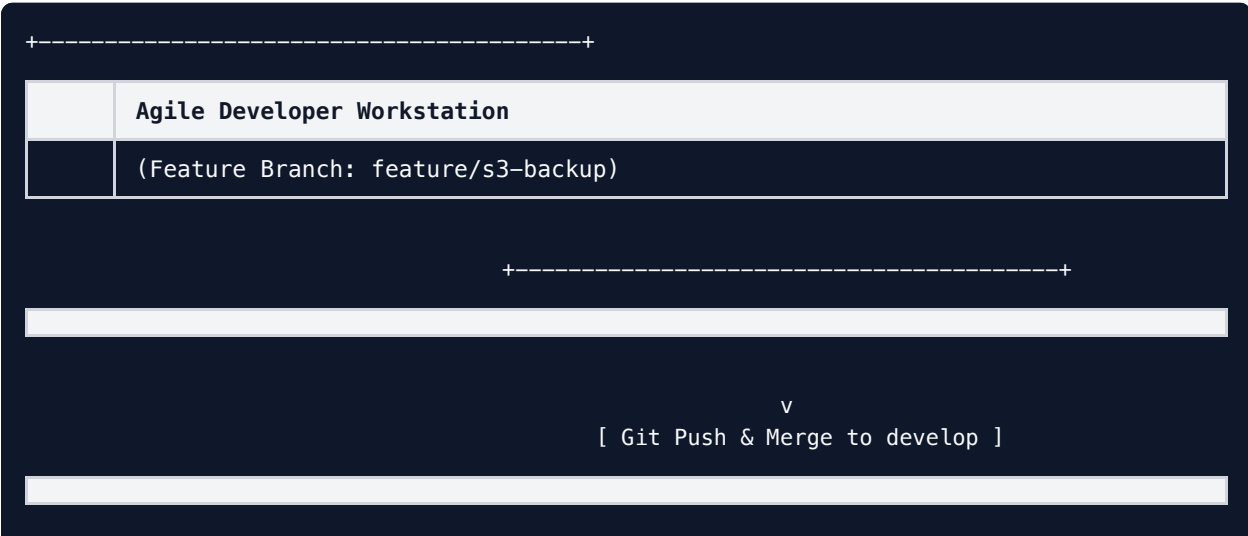
- **Author:** Avinash Agarwal
- **Repository:** [https://github.com/AVINASH1805A/AWS-PROJECT]
(https://github.com/AVINASH1805A/AWS-PROJECT)
- **Domain:** Cloud Computing, DevOps, Software Engineering & Agile Release Governance
- **Primary AWS Services:** Amazon EC2, Amazon S3, Amazon DynamoDB, AWS SSM Parameter Store, AWS IAM, Amazon CloudWatch, Amazon SNS
- **Tech Stack:** Node.js, Express, HTML5, CSS3 (Glassmorphic Theme), JavaScript (ES6+), Git, Shell Scripting

Executive Summary

This project presents an enterprise-grade cloud application and automated release deployment pipeline built on Amazon Web Services (AWS). Titled **BranchFlow**, the system combines a modern full-stack web application with dual cloud data persistence (**Amazon S3 & Amazon DynamoDB**), centralized configuration governance (**AWS Systems Manager Parameter Store**), infrastructure health monitoring (**Amazon CloudWatch & Amazon SNS**), keyless security authorization (**AWS IAM**), and strict Agile release governance across isolated **Development** and **Production** cloud environments on **Amazon EC2**.

The release workflow adheres to dual-branch Git governance (`develop` and `main`). Deployments to EC2 servers are fully automated using a custom shell script (`deploy.sh`), eliminating manual configuration errors and establishing zero-downtime release practices.

System Architecture





Key Modules & Implementation Details

1. Web Application Core & Cloud Persistence Engine (`server.js`)

* **Backend Framework:** Node.js with Express framework. * **API Endpoints:** * `GET /tasks` : Retrieves all active tasks and environment metadata. * `POST /tasks` : Adds a new task and triggers automated S3 backup & DynamoDB sync. * `PUT /tasks/:id` : Updates task completion status or details and syncs to cloud. * `DELETE /tasks/:id` : Deletes a task and updates S3 backup & DynamoDB. * `POST /backup` : Manually triggers an instant S3 & DynamoDB cloud sync. * `GET /backups` : Lists historical backup objects from S3. * `GET /health` : Diagnostic endpoint returning OS memory, CPU load, uptime, S3, DynamoDB, and SSM connectivity. * **AWS Cloud Integration Engine:** * **AWS S3 Backup Engine:** Utilizes `@aws-sdk/client-s3` v3 library. Asynchronously generates timestamped JSON files under `s3://branchflow-backups-avinash24/backups/tasks-.json`. * **AWS DynamoDB NoSQL Database:**

Utilizes `@aws-sdk/client-dynamodb` and `@aws-sdk/lib-dynamodb` to store live task items in the `BranchFlowTasks` table. * **AWS SSM Parameter Store:** Utilizes `@aws-sdk/client-ssm` to retrieve central environment configurations.

2. Modern Glassmorphic User Interface (`public/`)

* Built with semantic HTML5, CSS design tokens, HSL color palettes, and glassmorphism backdrop filters. * Displays live metric counters (Total Tasks, In Progress, Completed, Cloud Sync Engine Status). * Features dynamic cloud status badges (`S3 Active` , `DynamoDB Active` , `SSM Configured` , `DEV` vs `PROD`). * Includes toast notification alerts confirming S3 cloud backup & DynamoDB sync completion.

3. Automated Release Deployment Script (`deploy.sh`)

* Executable shell script residing on EC2 instances (`chmod +x deploy.sh`). * Parameterized execution (`./deploy.sh develop` for Dev, `./deploy.sh main` for Prod). * Automates: 1. Git fetch and checkout of target branch. 2. Dependency installation (`npm install --production`). 3. Graceful termination of older node processes (`kill -f`). 4. Background daemon startup (`nohup node server.js > app.log 2>&1 &`). 5. Deployment audit logging (`deploy.log`).

4. Cloud Infrastructure & Security Governance

* **Amazon EC2:** * `dev-server` (Ubuntu 22.04 LTS, `t2.micro`): Dedicated to testing incoming `develop` releases. * `prod-server` (Ubuntu 22.04 LTS, `t2.micro`): Dedicated to serving live `main` releases. * **AWS IAM Role (`EC2-S3-Access`):** * Grants `AmazonS3FullAccess` and DynamoDB/SSM permissions to EC2 instances using temporary AWS STS credentials. * **EC2 Security Groups:** * Port 22 (SSH): Restricted terminal access. * Port 3000 (Custom TCP): Web application access.

5. Observability & Infrastructure Monitoring

* **Amazon CloudWatch Alarm (`prod-server-high-cpu-alarm`):** * Monitors `CPUUtilization` metric on `prod-server` . * Threshold: Triggered when CPU utilization exceeds 70% over a 5-minute evaluation period. * **Amazon SNS (`ec2-high-cpu-alert`):** * Sends automated email alerts to system operators upon alarm state transition.

Agile Sprint 2 User Stories & Governance

Story ID	User Story	Priority	Points	Status
STORY-201	AWS S3 Backup: Backup task changes to S3 automatically.	High	5	Done
STORY-202	Release Script: Automate branch deployment on EC2 with <code>deploy.sh</code> .	High	3	Done
STORY-203	CloudWatch Monitoring: Configure CPU >70% alarm with SNS email alerts.	High	5	Done
STORY-204	Environment Diagnostics: Display DEV vs PROD badge and health statistics in UI.	Medium	2	Done
STORY-205	Keyless IAM Security: Attach IAM Role <code>EC2-S3-Access</code> to EC2 instances.	High	3	Done

Story ID	User Story	Priority	Points	Status
STORY-206	AWS DynamoDB Sync: Synchronize tasks to <code>BranchFlowTasks</code> NoSQL table.	High	5	Done
STORY-207	AWS SSM Parameter Store: Centralize environment parameters via SSM.	Medium	3	Done

Conclusion

The **BranchFlow** project successfully fulfills all requirements for an Agile-governed cloud release pipeline. By integrating AWS S3, AWS DynamoDB, AWS SSM Parameter Store, AWS CloudWatch, AWS SNS, AWS IAM, and shell script automation for EC2 deployments, the system provides a comprehensive, scalable, and secure cloud web application architecture suitable for production deployment.